

Campo IOAI

- **Limite de tempo:** 5 minutos
- **Armazenamento:** 5 GB
- **Tamanho da solução:** `solution.ipynb`, `custom_model.py` ≤ 1 MB em conjunto
- **Modelos pré-treinados:** nenhum — treinar de raiz, sem internet no momento da avaliação
- **Pontuação de referência:** 31.2187

Tarefa

O Presidente da Câmara de Astana pretende decorar a cidade com logótipos da IOAI estilizados. Enquanto estatístico, ele vê tudo — incluindo o logótipo — como uma função espacial $F(x, y, \overline{W})$, em que $x, y \in [0, 1]$ representam coordenadas num plano 2D e $\overline{W}$ é um conjunto de parâmetros ocultos que definem atributos estilísticos, tais como as cores e os ângulos das letras.

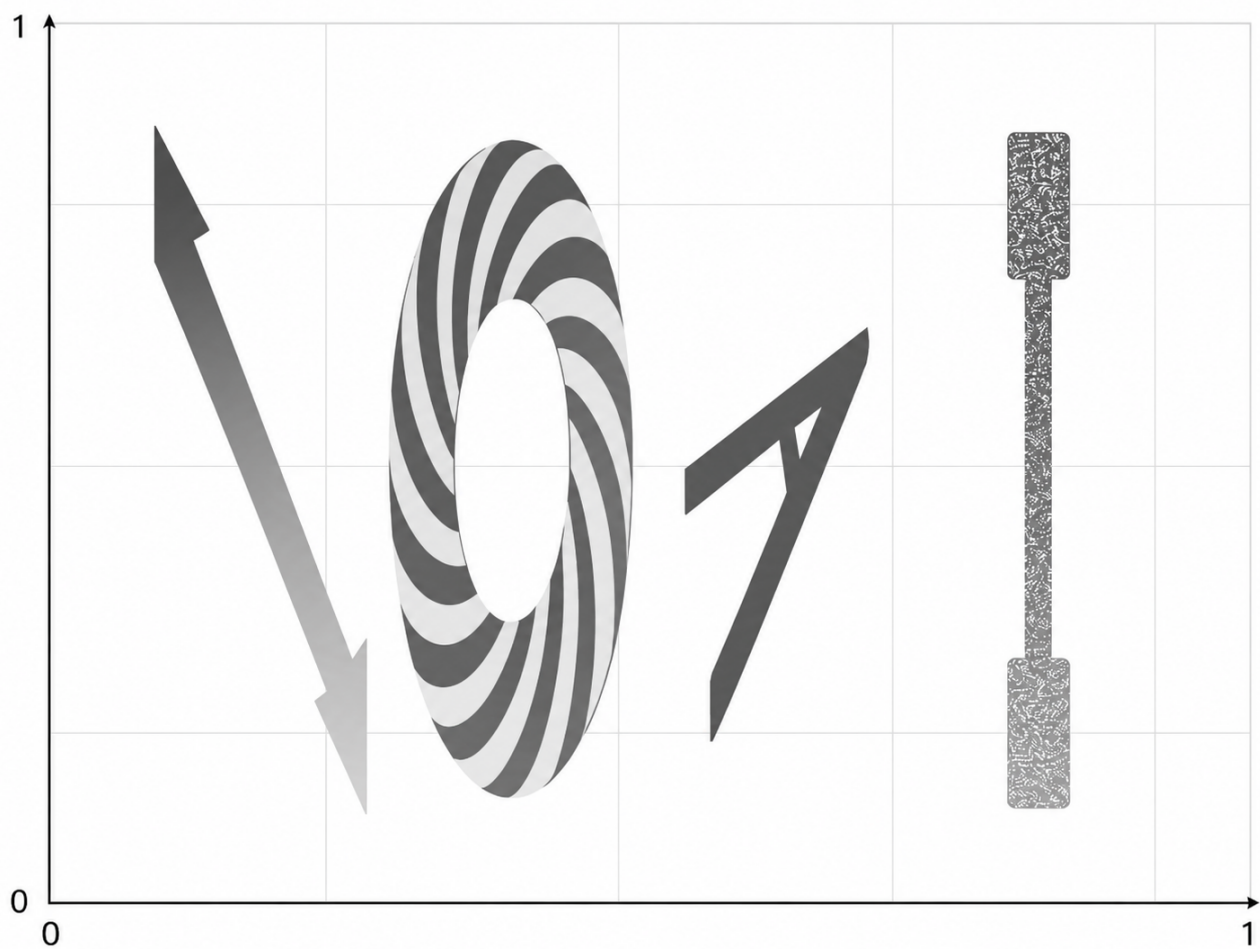
Uma vez que F é demasiado complexa para ser expressa como uma equação matemática explícita, a sua tarefa consiste em treinar uma rede neuronal para a aproximar. A rede produzirá um valor de **campo IOAI** para qualquer par de coordenadas (x, y) , gerando uma visualização completa do logótipo como mapa de calor em todo o plano. Eis um exemplo de uma visualização como mapa de calor de F com determinados parâmetros ocultos específicos $\overline{W}$.

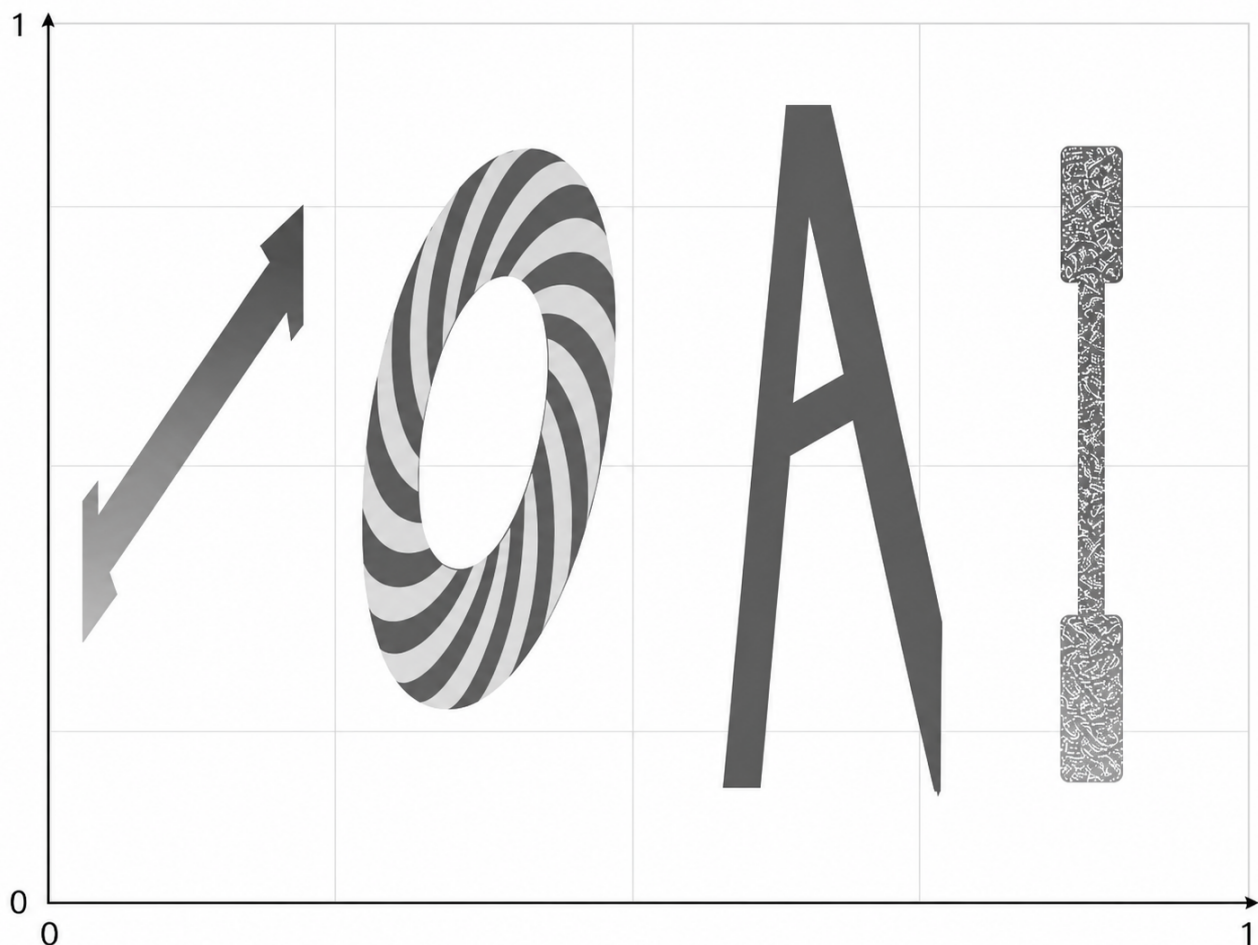


Em que consiste o campo IOAI? Quatro letras e o fundo.

- Os valores no interior da primeira letra $\mathbb{I}$ são muito elevados ($1e+10$ e superiores), com um gradiente linear
- Os valores na letra $\mathbb{O}$ apresentam um padrão em espiral
- Os valores dentro da letra $\mathbb{A}$ são sempre -1
- Os valores no interior da última letra $\mathbb{I}$ devem ser valores aleatórios no intervalo $[-2026, 2026]$, mesmo quando avaliados duas vezes no mesmo ponto
- Fora das letras, o valor é sempre zero

A função tem parâmetros ocultos $\overline{W}$, que afetam a escala e a inclinação das letras, juntamente com o intervalo de valores no interior da primeira letra $\mathbb{I}$. No entanto, as letras não se vão intersestar. Eis alguns exemplos ilustrativos do aspeto do campo IOAI com diferentes $\overline{W}$:





O que lhe é fornecido:

Este problema NÃO contém datasets. Em vez disso, é-lhe fornecida a função geradora, que é configurada pelo ficheiro de configuração JSON em `data/train_config/field_config.json`.

A configuração de teste está oculta, mas é de natureza semelhante. A sua tarefa consiste em fazer o ajuste ao gerador fornecido utilizando tantos dados quantos desejar. As suas distribuições de «treino» e de «teste» são geradas pelo mesmo gerador — simplesmente não sabe em que pontos (x_i, y_i) será avaliado.

A sua submissão deve ser composta por:

- a classe do modelo de treino guardada como `custom_model.py`. Este modelo deve herdar da classe `torch.nn.Module` e utilizar apenas imports de `torch`. Deve conter a classe `CustomModel` utilizada no notebook `solution.ipynb`.
- o notebook `solution.ipynb`, que produzirá os pesos `model.pt`

Pontuação

Para cada região, a pontuação mínima é 0 e a pontuação máxima é 1. A pontuação final é calculada como a média das cinco regiões (quatro, uma para cada letra, e o fundo) e multiplicada

por 100. Existe uma **penalização por número de parâmetros**:

Se o seu modelo tiver mais de 20260 parâmetros, a pontuação é reduzida a metade.

O número de parâmetros é medido por `sum(p.numel() for p in model.parameters())`. Esperamos que o seu modelo também funcione num modo estocástico, fazendo o `nn.Dropout` do PyTorch parte do modelo.

Para regiões padrão

Para cada região R (primeira letra `I`, `O`, `A`, `Background`), avaliamos o modelo em $N_R = 512$ pontos de teste (x_i, y_i) , com valores verdadeiros v_i e previsões $\hat{v}_i$. Utilizamos o erro absoluto médio (MAE) normalizado como métrica principal. O MAE é definido como:

$$\text{MAE}(R) = \frac{1}{N_R} \sum_{i=1}^{N_R} |\hat{v}_i - v_i|.$$

E a normalização é efetuada da seguinte forma:

$$\text{score}(R) = 1 - \min \left(\frac{\text{MAE}(R)}{s_R}, 1 \right),$$

em que $s_R > 0$ é uma constante de escala.

Para a região da última letra `I`

Nesta região, o **dropout está ativado durante a avaliação**. Para cada ponto de teste j :

1. Executamos o modelo $K = 10$ vezes para obter $\hat{v}_{j,1}, \dots, \hat{v}_{j,K}$.
2. Se alguma saída estiver fora do intervalo $[-2026, 2026]$, então $\text{pointScore}(j) = 0$.
3. Caso contrário, calculamos o desvio-padrão σ_j das K saídas e convertemo-lo numa pontuação:

$$\text{pointScore}(j) = \min \left(\frac{\sigma_j}{s_E}, 1 \right),$$

em que $s_E > 0$ é uma constante de escala fixa.

A pontuação da região é a média de todos os pontos da região:

$$\text{score}(I_{last}) = \frac{1}{N_E} \sum_{j=1}^{N_E} \text{pointScore}(j),$$

em que $N_E = K * N_R$.

Em termos simples, quanto maior for a diversidade, maior será a sua pontuação nesta região. **Não pode utilizar aleatoriedade em forma pura, incluindo as funções `rand*` e `_uniform` do**

PyTorch; a aleatoriedade deve resultar da inferência com o dropout ativado.

Como submeter

1. Abra `solution.ipynb` e execute todas as células.
2. Melhore o modelo `CustomModel` em `custom_model.py`
3. Certifique-se de que a última célula guarda o seu modelo no ficheiro `model.pt`.
4. No separador Git do JupyterLab, prepare, comente e faça commit de `solution.ipynb` e `custom_model.py` e, em seguida, faça push.
5. Regresse à página do concurso e clique em **Submit**. O comentário da submissão deve ser igual ao comentário do passo anterior.